Dynamic Field-of-View-Based Clustering for Efficient 360-degree Multicast Streaming

Ahmed Saadallah, Sidi-Mohammed Senouci, Ines El-Korbi, Philippe Brunet
Université de Bourgogne, DRIVE Lab, Nevers, France, email: {firstname.lastname}@u-bourgogne.fr

GLOBECOM 2024 - 2024 IEEE Global Communications Conference | 979-8-3503-5125-5/24/$31.00 ©2024 IEEE | DOI: 10.1109/GLOBECOM52923.2024.10901187

***Abstract*—The popularity of 360-degree video streaming has surged in recent years. However, delivering high-quality content with limited bandwidth poses a significant challenge, particularly in mobile delivery setups. Although techniques like predicting a user's Field of View (FoV) can improve streaming efficiency, multicasting offers a superior solution for delivering content to multiple users simultaneously. However, when users focus on different zones (FoV) in 360° videos, multicast streaming becomes difficult to implement effectively. To address this challenge, we propose in this paper a novel approach that leverages user-specific FoV prediction to dynamically adapt multicast streams. We propose a dynamic architecture that clusters users for optimized streaming based on their predicted FoV. This clustering leverages a DBSCAN-inspired algorithm that incorporates user head movement data. Users with similar FoV preferences are grouped, enabling transmission of only the relevant portions of the 360° video to each cluster. This significantly reduces bandwidth consumption compared to traditional per-user delivery, thus enhancing the overall Quality of Experience (QoE) for users.**

***Index Terms*—360-degree video streaming, Clustering, Multicasting, Tile-based, FoV.**

I. Introduction

The trend of streaming 360-degree videos has experienced a notable upswing in recent years, leading prominent technology platforms like YouTube, Meta, and Apple to adjust and provide their own solutions. Furthermore, numerous headset manufacturers have also jumped on board this trend [1]. As a result, it is expected that the user base for such systems will experience exponential growth in the years to come. A 360-degree streaming system typically involves a server broadcasting videos on demand to users wearing VR headsets. However, the complexity arises from the need for high-quality content, often at resolutions such as 8K. To address this challenge, a widely recognized method known as Field of View (FoV) prediction [2], [3] has been proposed in the literature. This method involves predicting the user's FoV in the upcoming seconds, thereby optimizing the streaming process. Despite such optimizations, challenges emerge when multiple users simultaneously request the same video. In such scenarios, the conventional per-user video delivery approach may become unreliable due to potential server congestion and user-side latency. To mitigate these issues, a multicasting solution can be employed, which involves broadcasting the same content to multiple users at the same time.

However, multicasting introduces its own set of challenges, especially when users have diverse FoV preferences within the 360-degree video. To tackle this issue, we present a novel approach in this paper: clustering users into groups to minimize the number of multicasting sessions required on the servers. This approach aims to optimize resources utilization and enhance the overall streaming experience, particularly the Quality of Experience (QoE) for users. Our proposed solution involves grouping users with similar content preferences and streaming videos to each cluster, ensuring they have access to the content they are interested in. Considering a large user base with shared network resources, the efficiency of our clustering algorithm becomes crucial. We opt for an algorithm inspired by the DBSCAN (Density-based Spatial Clustering of Applications with Noise) algorithm [4] for several reasons. Firstly, DBSCAN is known for its ability to handle noise and varying cluster shapes efficiently. This is important in our context as user preferences may not always neatly fit into predefined categories, and there may be outliers in the data. Secondly, this algorithm has a time complexity of $O(n \log n)$, making it suitable for processing large datasets without incurring excessive computational costs. By leveraging these characteristics, our algorithm can effectively partition users into clusters based on their content preferences while minimizing computational overhead, thus ensuring efficient resource utilization in the network. The clustering in our proposed approach is recalculated at variable intervals, allowing adaptation to user dissatisfaction, defined by factors such as the distance from the center of the Field of View in a cluster. We redefine the distance metric between users, incorporating similarity for more accurate clustering. The clustering interval is dynamically chosen based on user satisfaction. An inference time analysis demonstrates the efficacy of our proposed method in achieving efficient multicast transmission of 360-degree videos to very large user groups.

The primary contributions of our work can be outlined as follows:

- Proposal of a multicast architecture tailored for 360-degree videos alongside a dynamic clustering method for fields of view.
- Assessment of clustering quality using the Davies-Bouldin index [5], accompanied by theoretical complexity analysis.
- Evaluation of the scalability of the clustering approach by analyzing execution time and frequency of clustering calculation.

The rest of the paper is structured as follows. Section II presents a review of existing studies related to the multicasting of 360° video. Section III details the presentation of the multicast architecture. Section IV illustrates the proposed multicast content-based approach, which includes a novel clustering method tailored to the users' head movements in 360° video. Section V delves into the intricacies of implementation and the corresponding results. Finally, the paper concludes with Section VI.

II. Related Work

The main challenge in streaming 360-degree videos to multiple users concurrently lies in efficiently managing bandwidth. The conventional strategy of individually transmitting the video stream to each user escalates bandwidth requirements, potentially leading to network congestion. An effective solution to this dilemma is the adoption of well-known multicast transmission technology. This technique allows the simultaneous broadcasting of the same 360-degree video content to various users. By doing so, it significantly reduces the redundancy of data transmission, lowers the overall demand on bandwidth, and mitigates the risk of network congestion. This approach optimizes bandwidth usage, ensuring smoother and more reliable streaming experiences for users across the network. In the literature, the 360-degree video multicasting approaches can be classified into two categories: User Interaction and Head Movement-Based Approaches, and System and Optimization-Based Approaches.

Within the User Interaction and Head Movement-Based Approaches category, studies like [6] focus on predicting head orientation for viewers using devices equipped with sensors. These studies compare techniques such as linear regression and Long Short-Term Memory (LSTM) models to predict head movement. Additionally, they introduce conjoint viewport prediction to optimize bandwidth usage by multicasting shared video portions. These approaches aim to enhance the efficiency of 360-degree video streaming in wireless networks. Another study, [7], presents a solution for mobile multicast streaming of live 360-degree videos, addressing challenges in user grouping, quality representation selection, and energy efficiency. By utilizing a dynamic programming algorithm, users are divided into multicast groups, and resource blocks are allocated to maximize average bitrate while ensuring fairness. This system efficiently selects quality representations for tiles within resource constraints, thereby enhancing average viewport quality and minimizing spatial variance. Furthermore, energy consumption is minimized through burst transmission, strategically allocating bursts to resource blocks and utilizing the Discontinuous Reception Mechanism (DRX) for energy efficiency. Additionally, [8] introduces a method that combines 360-degree video features with limited FoV feedback for accurate prediction: Leveraging spherical convolution-based neural networks. This approach enhances prediction accuracy, which is crucial for multicast scenarios. The model integrates salient feature extraction, FoV prediction modules, and user feedback, culminating in accurate predictions through fusion techniques and weighted mean square error emphasis.

Within System and Optimization-based Approaches, studies such as [9] introduce an algorithm that employs dual problem decomposition. It proposes a distributed rate adaptation algorithm (DDA) that operates iteratively to achieve optimal user bitrates. Another study, [10], presents two methods for multicasting 360-degree videos, involving grouping either by viewer perspectives or by channel quality. These methods underscore the importance of user experience by considering variables such as video resolution and loading time.

The literature highlights several key problems with 360-degree video multicasting strategies. OScalability emerges as a primary concern due to frequent clustering, straining system capabilities to meet increasing user demands effectively. Moreover, inefficient distance calculation methods result in subpar performance, as they may not accurately gauge user proximity, leading to unnecessary grouping and resource allocation. Another challenge is the static nature of group identification, lacking dynamic adaptation over time to accommodate changes in user behavior or network conditions. This lack of adaptability limits the system's ability to optimize resources utilization and to deliver an optimal streaming experience to users. Our work aims to address these challenges by proposing a novel approach that incorporates dynamic clustering based on user behavior and network conditions. This approach mitigates scalability issues and significantly improves the efficiency of grouping users with similar content preferences. In the following sections, we delve deeper into the details of our proposed approach, including the clustering algorithm and its performance evaluation against existing methods.

III. System Architecture

In this section, we start by presenting an overview of the system architecture of our multicast content-based approach. Following that, we introduce the model for 360-degree video and discuss the various parameters considered.

A. System overview

Our system architecture depicted in Figure 1 represents a multicast transmission system for 360-degree video content, which utilizes dynamic group clustering presented in IV. The system begins with individual users in VR headsets predicting their long-term FoV [2]. Based on these predictions, each headset requests the corresponding video tiles, which are specific segments of the 360-degree video stream that fall within the user's expected FoV.

On the server side, the architecture is designed to optimize the streaming process. Users are clustered according to the patterns of their Head-Mounted Display (HMD) movements. This clustering is then used to determine the multicast groups, with each group containing users with similar viewing directions. The server prepares the video stream for each group by selecting the tiles that are in demand by the clustered users. After the selection process, the server initiates the multicast sessions. Each group receives a video stream that contains only the tiles relevant to its collective predicted FoV.

B. 360-degree Video modeling

To simplify the analysis of our study, we divide the 360-degree video into one-second segments, each containing 30

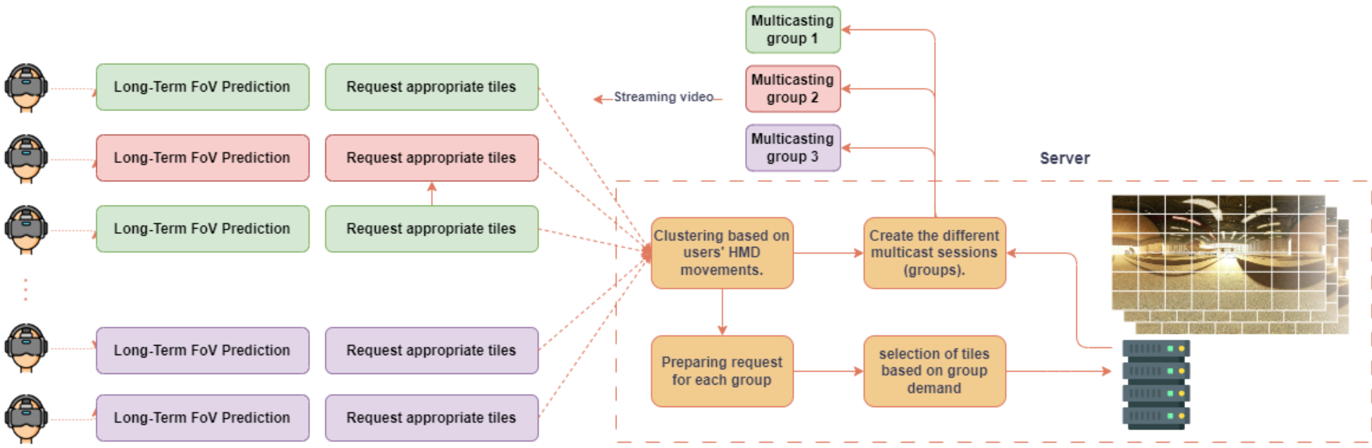


Fig. 1: System overview

frames. Each frame is initially projected onto an equirectangular projection. Following this projection, the frame is subdivided into multiple tiles arranged in an 8x6 grid.

Algorithm 1 Field of View Clustering Algorithm

Inputs:
- T: Threshold to consider points of view (FoV) in the same cluster.
- C_{FoV}: Center of the point of view defined by (X, Y).
- W: Window size, representing the number of C_{FoV}s.
- u_i: Time series representing the $W.FoVs$ of a user.
- n: Number of users.
- U: Set of users watching the same video, where $U = \{u_1, u_2, \ldots, u_n\}$.
- c: Set representing the current cluster.
- C: Set representing all clusters.

Output: Set C of all clusters

Sort U using an $O(n \log n)$ algorithm to obtain U_{sorted}.
Start with a user u_i such that $D(u_i, u_{i-1}) > T$.
Initialize current cluster c as empty.
Initialize the set of all clusters C as empty.
for each user u_i in U_{sorted} **do**
  if c is empty **then**
    Add u_i to c.
  end if
  if $D(c[0], u_i) \leq$ T **then**
    Compare T for each W.
    Identify the minimum number of compliant segments for dynamic clustering.
    Add u_i to c.
  else
    Add c to C and reset c as empty.
  end if
end for
if c is not empty **then**
  Add the last cluster c to C.
end if

Furthermore, we encode each tile at various quality levels. While our analysis focuses on two quality levels, it's worth noting that this approach could be generalized to accommodate more quality levels. Additionally, we assume that tiles within the Field of View (FoV) will be transmitted at a higher quality compared to tiles outside the FoV.

IV. Field of View Clustering Algorithm

Our clustering approach inspired from DBSCAN algorithm [4] is designed to group users watching the same 360-degree video based on similarities in their (FoVs), which are defined by their viewing direction coordinates (X, Y) projected in equirectangular projection. The algorithm requires a predefined threshold to determine whether FoVs belong to the same cluster, considering the window size W that represents the number of center Field of Views C_{FoVs}. It also involves a time series u_i representing the FoVs of a user within W, and n as the number of users, forming a set U of users watching the same video.

The process begins by sorting the users in U based on their FoV sequence to optimize the clustering process. Starting with an initial user u_i and an empty current cluster c, the algorithm iterates through the sorted users U_{sorted}. For each user, we check if their FoV is within the threshold distance from the first member of the current cluster c; if so, the user is added to c. If not, the current cluster c is considered complete, added to the set of all clusters C, and a new cluster c is initiated. This process repeats until all users are either assigned to a cluster or a new cluster is formed. The result is a set of clusters C, where each cluster contains users with similar FoVs, optimizing the delivery of 360-degree video content by minimizing bandwidth through multicast transmission to users within the same cluster.

To compare the similarity of users' FoVs, let's consider two sequences of C_{FoV} points, $(x_{1,i}, y_{1,i})$ and $(x_{2,i}, y_{2,i})$, for $i = 1, \ldots, W$, representing the positions of two users' focus over a window of W observations. The distance D between these sequences is calculated as:

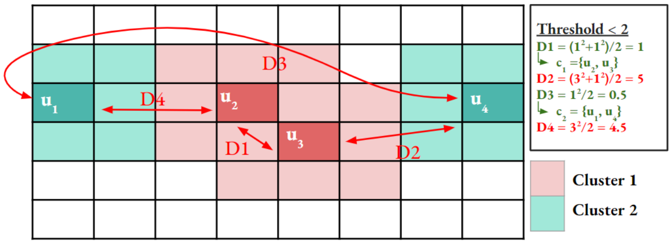


Fig. 2: Execution of algorithm 1 on a single frame

$$D = \sum_{i=1}^{W} \frac{d_x(x_{1,i}, x_{2,i})^2 + (y_{1,i} - y_{2,i})^2}{2W} \quad (1)$$

where $d_x(i, j)$ is defined as:

$$d_x(i, j) = \min(j - i, N - j + i) \quad \text{where} \quad i \leq j \quad (2)$$

and accounts for the cyclical or wrap-around nature of the indices.

In Figure 2, the algorithm 1 execution is depicted on a single frame featuring four distinct users. Following the arrangement of users along the x-axis, the selection of a user to start the iterations is conducted to ensure deterministic clustering. This selection process is guided by the criterion that the chosen user must exhibit a distance D greater than the threshold with its preceding neighbor, conforming to the condition $D(u_{i-1}, u_i) <$ Threshold. This approach serves to initialize the clustering process with precision, ensuring robust and consistent results.

Additionally, we conducted experiments utilizing a real video dataset [11]. As illustrated in Figure 3, our observations demonstrate the dynamic nature of our approach, wherein the number of clusters can be flexibly adjusted by modulating the threshold, enabling fine-tuning to suit specific requirements.

TABLE I: Information about the videos used in our experiments

Content	Duration	Category
Weekly Idol-Dancing	4:38	Performance
Damai Music Live-Vocie Toy	5:41	Performance
Rio Olympics VR Interview	3:07	Talkshow
Female Basketball Match	6:01	Sport
SHOWTIME Boxing	2:52	Sport
Gear 360: Anitta	3:25	Performance
CCTV Spring Festival Gala	6:13	Performance
VR Concert	3:46	Performance
Hey VR Interview	8:40	Talkshow

V. Performance Evaluation

In this section, we begin by providing details about the experimental setups. We then proceed to evaluate the quality of our clustering method in comparison to DBSCAN. Following that, we discuss the time complexity and scalability of our approach. Finally, we delve into the frequency of clustering calculations.

A. Experimental configurations

To simulate users' head movements, we chose several videos from the dataset referenced in [11]. These videos depict 48 users viewing various content. Table I provides information on the videos used in the experiments, along with recordings of participants' head movements. The selection and arrangement of tiles in the setup were executed as [12], resulting in the division of 360-degree videos into one-second segments comprising 8 rows and 6 columns of tiles.

B. Evaluation of the clustering method

To assess the quality of our clustering, we employ the Davies-Bouldin Index (DBI) [5]. The DBI quantifies the average *similarity* between each cluster and its closest counterpart. This similarity is determined through a function that considers both the distance between cluster centroids and the sizes of the clusters involved. Mathematically, the DBI is defined as:

$$DBI = \frac{1}{k} \sum_{i=1}^{k} \max_{j \neq i} \left(\frac{\sigma_i + \sigma_j}{D(c_i, c_j)} \right) \quad (3)$$

where k represents the number of clusters, σ_i denotes the average distance of all elements in cluster i to the centroid of that same cluster, and $D(c_i, c_j)$ refers to the distance between the centroids of clusters i and j. We compare our approach to DBSCAN in terms of DBI, as illustrated in Figures 4a and 4b. Our approach demonstrates lower Davies-Bouldin Index (DBI) scores compared to DBSCAN, consistently maintaining values below 1. This indicates significantly better clustering quality and greater control over user similarity in our approach. This improvement can be attributed to the different distance metrics employed: DBSCAN uses the Euclidean distance between two users, whereas our approach uses the distance D as defined in (1).

C. Time Complexity and Scalability Analysis

To assess the scalability of our approach, we conducted a comparative analysis against the Hierarchical algorithm, evaluating complexities and systematically increasing the number of users to a certain performance thresholds.

1) - Time Complexity:
The time complexity of the algorithm is dominated by the sorting operation. Since the algorithm sorts n users using an $O(n \log n)$ sorting algorithm, the time complexity of the sorting operation is $T(n) = O(n \log n)$.

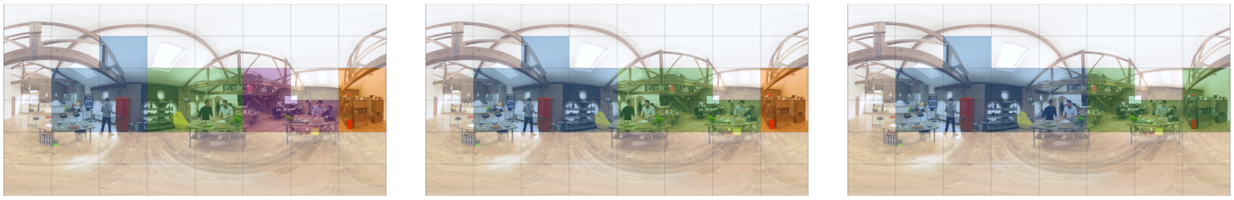
Inside the loop that iterates through n users, the most significant operations are constant time comparisons and adding elements to sets. These operations contribute $O(1)$ time complexity per iteration.

Therefore, the total time complexity of the algorithm is:

$$T(n) = O(n \log n)(\textit{sorting}) + O(n)(\textit{loop iterations})$$

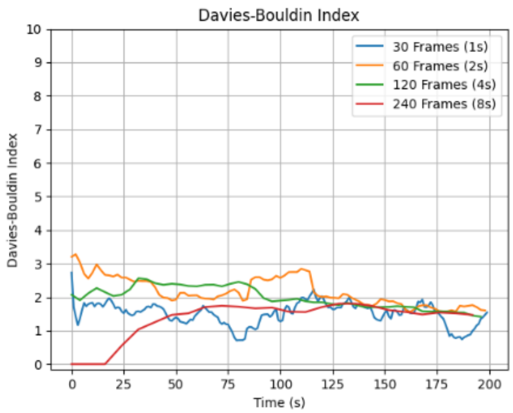
In most cases, the sorting operation dominates the time complexity, making the overall complexity $O(n \log n)$.

Hierarchical clustering is one of the most widely used algorithms among clustering techniques. Its ability to create

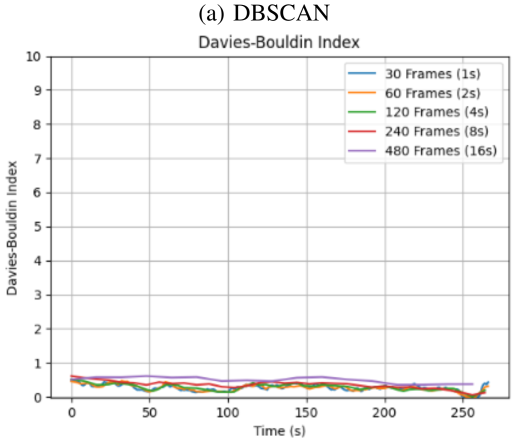


$Threshold = 2$ (4 clusters) $Threshold = 4$ (3 clusters) $Threshold = 6$ (2 clusters)

Fig. 3: Comparison between the different threshold configurations for single-frame clustering.



(a) DBSCAN



(b) Proposed approach

Fig. 4: The Davies-Bouldin Index across different window sizes for DBSCAN and our proposed approach.

hierarchical structures of clusters provides valuable insights into the relationships between data points. The hierarchical nature of this clustering method allows for a flexible exploration of the data's underlying structure, which can be particularly useful in exploratory data analysis and understanding complex datasets.

The problem with this clustering is that it is more complex than our proposed clustering approach, since it has complexity $O(n^3)$. This is due to the pairwise distance computation for all data points, resulting in high computational overhead especially for large datasets. As the number of data points increases, the time and memory requirements grow significantly, making hierarchical clustering impractical for big data applications or real-time processing tasks.

Our approach therefore represents a good alternative for faster, scalable, and efficient clustering.

TABLE II: Comparative complexity analysis

Clustering Method	Time Complexity	Space Complexity
Our approach	$O(n \log n)$	$O(n)$
DBSCAN	$O(n \log n)$	$O(n)$
Hierarchical	$O(n^3)$	$O(n^2)$

2) - Scalability and Performance Limit:
In order to test the scalability of our approach, we opt to expand the dataset by employing data augmentation techniques such as duplication, interpolation, and noise injection. These methods allow us to generate additional data points while preserving the characteristics and distribution of the original dataset. By augmenting the data, we could simulate a larger number of users or instances, providing a more comprehensive evaluation of our clustering algorithm's performance under variable scales. We systematically increase the size of the dataset until reaching a point where the clustering calculation could be completed within a reasonable timeframe. This process involves iteratively enlarging the dataset and measuring the execution time of the clustering algorithm. Our aim is to find the threshold where the algorithm could process the data efficiently without compromising accuracy or exceeding acceptable computational resources.

As illustrated in Figure 5, our algorithm demonstrated remarkable scalability, achieving linear performance as the number of users increase. We successfully clustered over 25,000 users' windows with a size of 30 frames (one second).

This indicates the robustness and efficiency of our approach, as it could handle a significant volume of data while maintaining acceptable computational performance. Such scalability is essential for real-world applications where large datasets are common, ensuring that our clustering solution remains effective and practical in various contexts.

D. Frequency of Clustering Calculations

In order to analyse the frequency of clustering application during a video, we recorded the instances of clustering recalculation. In traditional approaches, clusters are computed for each segment (each second), necessitating readjustment every second. However, this approach may not be optimal, as clusters

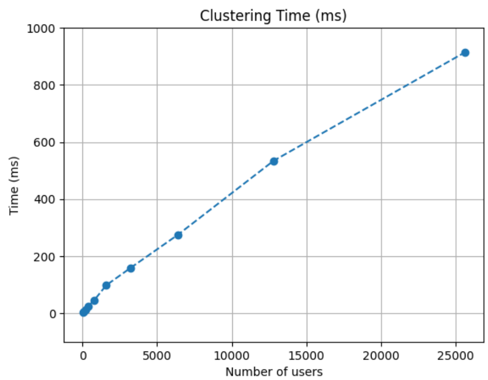


Fig. 5: Variation in clustering execution time with different numbers of users

sometimes remain stable for several seconds, rendering recalculations unnecessary. Hence, we adopted dynamic clustering, where recalculations occur only as needed. Figure 6 illustrates the frequency of clustering application throughout the video. It indicates that clustering is applied just over 80 times at a one-second frequency over a video lasting 3 minutes and 20 seconds. This translates to a calculation rate of approximately 40% at this frequency (83 calculations out of 205). This significantly reduces the computational cost compared to the pre-existing approach, which applies clustering every second throughout the video.

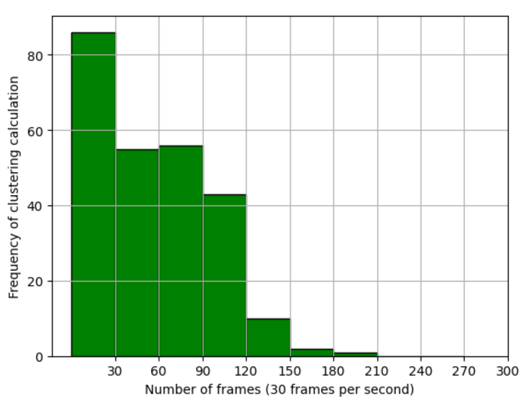


Fig. 6: Frequency of clustering calculations for 20 users watching 3 different videos with a threshold of 3.

VI. Conclusion

In this paper, we propose a novel approach for multicast transmission of 360-degree videos to large user groups. The growing popularity of 360-degree video streaming creates challenges due to the high bandwidth requirements. Existing solutions often employ per-user video delivery, leading to network congestion. This paper addresses this challenge by proposing a multicast architecture that utilizes dynamic clustering of users based on their predicted Field of View (FoV). Users with similar FoV preferences are grouped together, and only the relevant video tiles are transmitted to each group, in order to reduce bandwidth consumption. The core contribution of this work is a clustering algorithm inspired by DBSCAN that considers user head movements. This algorithm efficiently groups users based on FoV similarity. We evaluate the proposed approach by comparing its clustering quality and scalability to existing methods. The results demonstrate that the proposed approach achieves lower Davies-Bouldin Index (DBI) scores, indicating superior clustering quality. Additionally, the time complexity analysis reveals a favorable $O(nlogn)$ complexity compared to the $O(n^3)$ complexity of hierarchical clustering, making it suitable for large user groups.

Acknowledgment

This work was supported by the VISIO3D+ project, which is partially funded by FEDER (European Regional Development Fund) and BPIFRANCE.

References

[1] T. Alsop, “Virtual reality (vr) software b2c market revenue in europe from 2018 to 2028, by segment,” Statista, 2024. [Online]. Available: https://www.statista.com/forecasts/1340254/vr-software-b2c-market-revenue-by-segment-europe

[2] C. Li, W. Zhang, Y. Liu, and Y. Wang, “Very long term field of view prediction for 360-degree video streaming,” in *2019 IEEE Conference on Multimedia Information Processing and Retrieval (MIPR)*. IEEE, 2019, pp. 297–302.

[3] A. Saadallah, I. El-Korbi, S. M. Senouci, and P. Brunet, “A new time series forecasting approach using classification: Application to field of view prediction in 360° videos,” in *2023 IEEE 97th Vehicular Technology Conference (VTC2023-Spring)*, 2023, pp. 1–5.

[4] X. Yu, D. Zhou, and Y. Zhou, “A new clustering algorithm based on distance and density,” in *Proceedings of ICSSSM ’05. 2005 International Conference on Services Systems and Services Management, 2005.*, vol. 2, 2005, pp. 1016–1021 Vol. 2.

[5] D. L. Davies and D. W. Bouldin, “A cluster separation measure,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. PAMI-1, no. 2, pp. 224–227, 1979.

[6] H. Cao, J. Lu, and N. Zong, “A multi-user 360-video streaming system for wireless network,” in *Proceedings of the 17th International Conference on Virtual-Reality Continuum and Its Applications in Industry*, ser. VRCAI ’19. New York, NY, USA: Association for Computing Machinery, 2019. [Online]. Available: https://doi.org/10.1145/3359997.3365713

[7] O. Eltobgy, O. Arafa, and M. Hefeeda, “Mobile streaming of live 360-degree videos,” *IEEE Transactions on Multimedia*, vol. 22, no. 12, pp. 3139–3152, 2020.

[8] J. Li, L. Han, C. Zhang, Q. Li, and Z. Liu, “Spherical convolution empowered fov prediction in 360-degree video multicast with limited fov feedback,” 2022.

[9] L. Zhong, M. Wang, C. Xu, S. Yang, and G.-M. Muntean, “Decentralized optimization for multicast adaptive video streaming in edge cache-assisted networks,” *IEEE Transactions on Broadcasting*, vol. 69, no. 3, pp. 812–822, 2023.

[10] J. Park, “Multi-session multicasting for 360-degree video multicast over ofdma systems,” *International Journal of Digital Multimedia Broadcasting*, vol. 2021, pp. 1–19, 2021.

[11] C. Wu, Z. Tan, Z. Wang, and S. Yang, “A dataset for exploring user behaviors in VR spherical video streaming,” in *Proceedings of the 8th ACM on Multimedia Systems Conference*, 2017, pp. 193–198.

[12] A. Saadallah, P. Brunet, I. El-Korbi, S.-M. Senouci, and S. Cherkaoui, “Leveraging graph theory for efficient cache policy design in 360° video streaming,” in *GLOBECOM 2023 - 2023 IEEE Global Communications Conference*, 2023, pp. 976–981.